

TagScan

Raspberry Pi Device Setup Guide

Step-by-step instructions to connect a Raspberry Pi scanner device to TagScan, so it can push its CSV scan files to the backend over HTTPS.

Generated: 2026-09-12

Module: TagScan (module-1)

Audience: whoever sets up and maintains the Raspberry Pi devices

What you will do in this guide

- Flash and boot a Raspberry Pi, and connect to it over SSH.
- Install the TagScan watcher script and its systemd service.
- Generate a device API key and copy the upload URL from the TagScan web app.
- Configure the watcher and start it as a background service.
- Confirm a real CSV file makes it from the Pi into TagScan.
- Point your own CSV-producing app at the watch folder.
- Set up separate dev and prod instances on the same Pi, if needed.
- Know where to look when something goes wrong.

NOTE

This guide assumes the TagScan backend is already deployed and reachable over HTTPS, and that you (or an administrator) have a super-admin or TagScan-admin login to the RW Crew web app. It does not cover deploying the backend itself - see docs/deploying-to-hostinger.pdf in the project repository for that.

1. Overview: how this works

The application running on the Raspberry Pi writes its scan results as CSV files into a local folder on the Pi. A small background program (the "watcher") keeps an eye on that folder. Whenever it sees a finished CSV file, it sends that file to the TagScan backend over HTTPS, using a secret key that identifies this specific device. Once the backend confirms it received the file, the watcher moves it into a "sent" folder on the Pi so it is never sent twice.

This means the Pi never needs direct access to the backend's disk or database - it only ever makes a single, ordinary HTTPS request per file. If the Pi loses its internet connection for a few hours, files simply pile up in the watch folder and get sent automatically once the connection comes back.

The three things every device needs

Item	What it is	Where it comes from
Upload URL	The web address the Pi sends files to.	TagScan web app, Settings screen.
API key	A secret password that identifies this one device.	TagScan web app, Scanners screen (per scanner).
Watch folder	The local folder on the Pi the watcher checks for finished CSV files.	Chosen by you, e.g. "/home/pi/tagscan"

IMPORTANT

Each TagScan environment (a "test"/"dev" deployment and a "production" deployment) has its own separate database, its own separate list of registered scanners, and therefore its own separate API keys. A key generated on the dev environment's Scanners screen will NOT work against the production backend, and vice versa. Section 9 of this guide explains how to run both side by side on one Pi safely.

2. What you need before you start

- A Raspberry Pi (3, 4 or 5) with a power supply and a micro-SD card (8 GB or larger).
- A Windows, Mac or Linux computer to prepare the SD card and connect over SSH.
- A network connection for the Pi (Wi-Fi details, or an Ethernet cable).
- A login to the TagScan web app with access to the Scanners and Settings screens (super-admin, or a TagScan role with view/edit permission on both).
- The TagScan backend's public web address (e.g. <https://your-domain.example.com>), already reachable over HTTPS from the internet.

3. Prepare and boot the Raspberry Pi

Step 3.1: Install Raspberry Pi Imager

On your own computer (not the Pi), download and install "Raspberry Pi Imager" from raspberrypi.com/software. Insert the SD card into your computer.

Step 3.2: Flash Raspberry Pi OS

Open Raspberry Pi Imager. Choose your Pi model, choose "Raspberry Pi OS (64-bit)" (the "Lite" version, without a desktop, is enough - this device only runs one background program), and choose your SD card as the storage target.

Step 3.3: Pre-configure it with the gear icon (very important)

Before clicking "Write", click the gear/settings icon (or press Ctrl+Shift+X) to open the advanced options, and set:

- Hostname - e.g. tagscan-pi-01 (helps you tell devices apart later).
- Enable SSH - tick "Enable SSH", choose "Use password authentication" (or paste your own SSH public key if you already use SSH keys).
- Username and password - e.g. username "pi", and a strong password you will remember.
- Wi-Fi (if not using a network cable) - enter your Wi-Fi network name and password, and set the correct Wi-Fi country.
- Locale settings - set your time zone and keyboard layout.

Save these settings, then click "Write" and let it finish (and verify, if offered).

Step 3.4: First boot

Put the SD card in the Raspberry Pi and power it on. Wait about a minute for the first boot to finish and for it to join your network.

Step 3.5: Connect over SSH

From your own computer's terminal (on Windows, use PowerShell or the Bash tool's terminal), connect using the hostname you set, or the Pi's IP address if the hostname does not resolve:

```
ssh pi@tagscan-pi-01.local
# or, if that name doesn't resolve:
ssh pi@<the-pi's-ip-address>
```

NOTE

You can find the Pi's IP address from your router's device list, or by running "ping tagscan-pi-01.local" from your computer.

4. Update the system and check Python

Step 4.1: Update the OS

Once connected over SSH, run:

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

Wait about 30 seconds and reconnect over SSH after the reboot.

Step 4.2: Confirm Python 3 is available

Raspberry Pi OS ships with Python 3 already installed. Confirm it:

```
python3 --version
```

The watcher script only uses Python's standard library, so no extra packages need to be installed with pip - this step is just a sanity check.

5. Install the watcher script

The watcher script and its systemd service file live in the RW Crew repository under `scripts/pi-watcher/`. You need to get two files onto the Pi: `watcher.py` and `tagscan-watcher@.service`.

Step 5.1: Create the installation folders

On the Pi, run:

```
sudo mkdir -p /opt/tagscan-watcher
sudo mkdir -p /etc/tagscan-watcher
```

Step 5.2: Copy the files onto the Pi

From your own computer (in a terminal, in the RW Crew repository folder), copy the two files to the Pi over SSH - replace "tagscan-pi-01.local" with your Pi's hostname or IP address:

```
scp scripts/pi-watcher/watcher.py pi@tagscan-pi-01.local:/tmp/watcher.py
scp scripts/pi-watcher/tagscan-watcher@.service pi@tagscan-pi-01.local:/tmp/tagscan-watcher@.service
```

Then, back on the Pi (over SSH), move them into place:

```
sudo mv /tmp/watcher.py /opt/tagscan-watcher/watcher.py
sudo mv /tmp/tagscan-watcher@.service /etc/systemd/system/tagscan-watcher@.service
```

NOTE

No internet connection on the Pi? You can also open `watcher.py` on your own computer, copy its text, and paste it into a new file on the Pi with a text editor ("`sudo nano /opt/tagscan-watcher/watcher.py`"), then do the same for the `.service` file.

Step 5.3: Create a dedicated, unprivileged user to run it

The watcher does not need admin rights - give it its own user account:

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin tagscan-watcher
```

6. Get the device API key and upload URL from TagScan

This step happens in your web browser, on the TagScan web app - not on the Pi. You will need a login with access to TagScan's Scanners and Settings screens.

Step 6.1: Register this device as a Scanner (if it isn't already)

Log in to the RW Crew web app, open "Modules" > "TagScan", and in the left-hand menu go to "Scanners" (under MasterData). If this Raspberry Pi is not listed yet, click "New scanner" and fill in its name (e.g. "Scan-01"), its Type, and its Technology ("Raspberry Pi 3", "4" or "5", matching your device).

Step 6.2: Generate an API key for this scanner

In the Scanners table, find this device's row and click the key icon in the Actions column. In the dialog that opens, click "Generate API key" (or "Generate new key" if one already exists - this immediately invalidates the old one).

IMPORTANT

The key is shown in plain text exactly once, right after you generate it. Copy it immediately (use the "Copy" button next to it) and paste it somewhere safe for the next step - the web app can never show it to you again. If you lose it, just generate a new one.

Step 6.3: Copy the device upload URL

Still in TagScan's left-hand menu, go to "Settings". Copy the value shown under "Device upload URL" using its "Copy" button. It will look like:

```
https://your-domain.example.com/api/public/tagscan-intake
```

You now have the two secrets this device needs: the API key from step 6.2, and the upload URL from step 6.3. Keep both handy for the next section.

7. Configure the watcher

Step 7.1: Create the local watch/sent folders

Decide where the CSV-producing app on this Pi should write its finished files, and where successfully-sent files should be moved to. A simple, recommended layout:

```
sudo mkdir -p /home/pi/tagscan/prod/outgoing
sudo mkdir -p /home/pi/tagscan/prod/sent
sudo chown -R tagscan-watcher:tagscan-watcher /home/pi/tagscan
```

Step 7.2: Create the environment file

Create /etc/tagscan-watcher/prod.env with a text editor:

```
sudo nano /etc/tagscan-watcher/prod.env
```

Paste in the following, replacing API_URL and API_KEY with your own values from Section 6:

```
API_URL=https://your-domain.example.com/api/public/tagscan-intake
API_KEY=module1_2_PASTE-YOUR-OWN-GENERATED-KEY-HERE
WATCH_DIR=/home/pi/tagscan/prod/outgoing
SENT_DIR=/home/pi/tagscan/prod/sent
POLL_INTERVAL_SECONDS=15
```

Save and exit (in nano: Ctrl+O, Enter, then Ctrl+X). Then lock the file down, since it contains a secret key:

```
sudo chown tagscan-watcher:tagscan-watcher /etc/tagscan-watcher/prod.env
sudo chmod 600 /etc/tagscan-watcher/prod.env
```

Setting	Meaning
API_URL	The upload address you copied in step 6.3.
API_KEY	The device's secret key you copied in step 6.2.
WATCH_DIR	Folder the watcher checks for finished *.csv files.
SENT_DIR	Folder successfully-uploaded files are moved into.
POLL_INTERVAL_SECONDS	How often (in seconds) to check WATCH_DIR. 15 is a good default.

8. Install and start the background service

The watcher runs as a "systemd" service, which is the standard way Linux runs background programs. This means it starts automatically on every boot, and restarts itself automatically if it ever crashes.

Step 8.1: Reload systemd and start the service

```
sudo systemctl daemon-reload
sudo systemctl enable --now tagscan-watcher@prod
```

The "@prod" part matches the "prod.env" file you created in step 7.2 - it tells systemd which environment file this particular instance of the watcher should use.

Step 8.2: Confirm it is running

```
sudo systemctl status tagscan-watcher@prod
```

You should see "active (running)" in green. Press "q" to exit the status view. You can watch its live log output with:

```
sudo journalctl -u tagscan-watcher@prod -f
```

The first line it logs should look like:

```
Watching /home/pi/tagscan/prod/outgoing, sending to https://your-domain.example.com/api/public/tag
```

Press Ctrl+C to stop following the log (this does not stop the service itself).

9. Test the connection end-to-end

Step 9.1: Drop a test CSV file into the watch folder

From the Pi's own terminal, create a small test file directly inside WATCH_DIR:

```
printf "EPC,RSSI\nABC123,70\n" > /home/pi/tagscan/prod/outgoing/test-scan.csv
```

Step 9.2: Watch it get picked up

Within about POLL_INTERVAL_SECONDS (15 seconds by default), the log should show a line like:

```
Uploaded test-scan.csv -> HTTP 201  
Moved test-scan.csv -> /home/pi/tagscan/prod/sent/test-scan.csv
```

The file should now be gone from the "outgoing" folder and present in the "sent" folder - check with:

```
ls /home/pi/tagscan/prod/outgoing  
ls /home/pi/tagscan/prod/sent
```

Step 9.3: Confirm it arrived in TagScan

Back in the TagScan web app, open "CSV Source Files" (the Dashboard screen) and browse into the "Unreaded Tags" folder - test-scan.csv should be listed there. From there, the normal "Tag Headerdata" screen's Scan button will log it into the database exactly like any other incoming file.

NOTE

The device's Scanners row also now shows a recent "last used" time next to its API key, confirming this specific device successfully authenticated.

10. Point your CSV-producing app at the watch folder

Now that the test file worked, configure the real application on the Pi that produces the scan CSV files to write its output into the WATCH_DIR you configured in step 7.1 (/home/pi/tagscan/prod/outgoing in this guide).

IMPORTANT

Do not have your application write the final .csv file directly, in place. Instead, have it write to a temporary name (e.g. "scan-0042.csv.tmp") and only rename() it to the final "scan-0042.csv" name once the file is completely finished. This is what stops the watcher from ever picking up a half-written file - the watcher only considers a file "ready" once its filename ends in .csv and its size has stopped changing between two checks.

11. Running a dev and a production instance on one Pi

If this Pi needs to feed both a test/dev TagScan deployment and the real production one (for example while validating a change before it goes live), repeat Sections 6 through 8 a second time using the instance name "dev" instead of "prod" throughout: create `/etc/tagscan-watcher/dev.env` with the dev environment's own `API_URL` and `API_KEY`, its own `WATCH_DIR/SENT_DIR` pair (e.g. `/home/pi/tagscan/dev/outgoing` and `.../sent`), and enable it as its own service:

```
sudo systemctl enable --now tagscan-watcher@dev
```

Both instances then run side by side, completely independently, each with its own restart cycle and its own log stream:

```
sudo systemctl status tagscan-watcher@dev
sudo systemctl status tagscan-watcher@prod
sudo journalctl -u tagscan-watcher@dev -f
```

IMPORTANT

Never paste a production API key into `dev.env`, or a dev API key into `prod.env`. Dev and production are separate databases with separate registered scanners, so a key from one will simply be rejected (401 Unauthorized) by the other - but keeping them cleanly separated avoids any confusion about which files went where.

12. Troubleshooting

Symptom	Likely cause	What to do
Service shows "failed" in system log	Missing/misspelled env file, or a typo in the env file path	Run "sudo journalctl -u tagscan-watcher@prod" to check the logs. The last entry should contain the error message.
Log shows "Missing required environment variable"	The env file is missing a required value or systemd can't find it.	Check the env file path in the service file. The env file should contain API_URL, API_KEY, WATCH_DIR and SECRET_DIR, each exists.
Upload fails with HTTP 401	Wrong, revoked, or expired API key	Check the API key in the env file. It should be a valid API key.
File never leaves the outgoing queue	The file's size is not changing (still being written)	Check the outgoing queue path in the env file. It should be a valid path.
Connection refused / timeout in the log	The Pi has the service set to the backend/its IP address	Check the backend IP in the env file. It should be the correct IP address.
Upload fails with HTTP 400	The producing app is writing a file with the same name as the file already in the queue	Check the file name in the env file. It should be a unique name.
"Object is already accepted"	When file is larger than the backend's configured upload limit.	Check the backend's upload limit settings. (Check with your backend administrator - the limit is set in the backend's own settings.)

Useful commands

```
# Restart a service after changing its env file
sudo systemctl restart tagscan-watcher@prod

# Stop / start a service
sudo systemctl stop tagscan-watcher@prod
sudo systemctl start tagscan-watcher@prod

# Live log, most recent lines first
sudo journalctl -u tagscan-watcher@prod -n 100 --no-pager

# Confirm the backend is reachable at all
curl -I https://your-domain.example.com/api/health
```

13. Security notes

- Treat an API key exactly like a password - anyone who has it can push files as this device. Store env files with the 600 permissions shown in step 7.2, and never commit them to git.
- If a Pi is lost, stolen, or decommissioned, revoke its API key immediately from the Scanners screen's key icon ("Revoke key") rather than leaving it active.
- Rotating a key ("Generate new key") immediately invalidates the previous one - update the env file and restart the service in the same session so the device doesn't sit unable to upload.
- The upload endpoint only ever accepts .csv files below the configured size limit, and never gives the device any access beyond dropping files into its own intake folder - a compromised device cannot read or modify anything else in TagScan.